

Database Module

Full-Stack Development

Relational vs. NoSQL Databases

Relational databases (MySQL, PostgreSQL) store data in tables with a fixed schema and strong relationships enforced between them via foreign keys — ideal when data is structured, relationships matter (a user has many orders, an order has many line items), and consistency is critical. NoSQL databases (MongoDB being the most common document-oriented example) store more flexible, often nested JSON-like documents without a rigid shared schema — useful when data is naturally hierarchical, schema needs to evolve quickly, or the access pattern is simple key-based lookups at large scale. Most real products default to a relational database unless they have a specific reason not to, since strong relationships and consistency guarantees are hard to get back once you give them up.

Relational Schema Design

Designing a good relational schema means modeling entities as tables and their relationships correctly:

- One-to-many — e.g. one college has many students; the 'many' side (students) holds a foreign key pointing back to the 'one' side (college).
- Many-to-many — e.g. students and courses (a student takes many courses, a course has many students) — modeled with a separate join table holding pairs of foreign keys.
- Normalization — organizing tables to avoid storing the same fact in multiple places, so an update only ever needs to happen in one row. Some duplication is occasionally accepted deliberately for performance — that's a tradeoff to make consciously, not by accident.

Query Design

Writing a query well means retrieving exactly the data needed, in the shape needed, without over-fetching. JOINS combine rows from related tables based on a shared key (an INNER JOIN returns only matching rows across both tables; a LEFT JOIN keeps all rows from the first table even when there's no match in the second). WHERE filters rows before grouping; HAVING filters after a GROUP BY aggregation. A common beginner mistake is fetching entire tables into application code and filtering there — that work almost always belongs in the query itself, which the database is optimized to do far faster.

Indexing & Performance Basics

An index is a separate, ordered data structure the database maintains alongside a table specifically to make lookups on certain columns fast — similar to an index at the back of a book. Without an index on a column used in a WHERE clause or JOIN, the database may have to scan every row in the table to find matches, which gets slow as data grows. The tradeoff: indexes speed up reads but slightly slow down writes (since the index has to be updated too) and use extra storage — so they're added deliberately on columns that are actually queried often, not on every column by default.

ORMs

An Object-Relational Mapper (ORM) — like Prisma, Sequelize, or Hibernate — lets application code interact with a relational database using the programming language's own objects and methods instead of writing raw SQL strings everywhere. ORMs typically also manage migrations (versioned, repeatable changes to the database schema as the application evolves) so schema changes can be tracked, reviewed, and applied consistently across development, staging, and production databases. ORMs remove a lot of repetitive SQL, but understanding the actual SQL an ORM generates underneath still matters — it's what you'll need to reason about when a query is slower than expected.

Learn More

MySQL Official Documentation

<https://dev.mysql.com/doc/>

MongoDB Official Documentation

<https://www.mongodb.com/docs/>

Prisma Official Documentation

<https://www.prisma.io/docs>